

Omokai Robotics Pipeline

README

A simulated drone pipeline: a natural language prompt goes to an LLM planner, the planner's output is checked by a validator before anything executes, and only a validated mission ever reaches the executor, which drives PX4 SITL over MAVSDK.

```
prompt -> LLM planner (proposes only) -> validator (guardrail) -> executor (MAVSDK) ->
PX4 SITL + Gazebo
```

One simulator, two build paths

Both the native setup and the Docker image run **Gazebo Harmonic** (gz-sim 8.14.0) against PX4's **gz_x500** target. They are not two different simulators the difference is reproducibility, not the underlying engine.

The native path builds PX4 directly on the host with `Tools/setup/ubuntu.sh` and `make px4_sitl gz_x500`. It's the fastest way to get a working simulator on a machine you already control, but it isn't pinned running the setup script again later could pull a newer PX4 or Gazebo release than what's documented here.

The Docker path exists to remove that variability: `docker/Dockerfile.px4` clones PX4 at one fixed commit and installs one fixed **gz-harmonic** package version, so the container is the same build every time regardless of who runs it or when.

Requirements

Ubuntu	22.04 LTS	what this was built and tested on
Python	3.10	ships with Ubuntu 22.04
PX4-Autopilot	cloned locally	native path only, at <code>~/PX4-Autopilot</code>
Gazebo Harmonic	gz-sim 8.14.0	installed by <code>Tools/setup/ubuntu.sh</code> , native path only
MAVSDK-Python	3.10.2	<code>pip3 install mavsdk==3.10.2</code>
Docker	24+	optional builds PX4 + Gazebo Harmonic from source, no host install needed

Quick start native

1. Install

```
git clone https://github.com/your-org/omokai-robotics.git
cd omokai-robotics
pip3 install -r requirements.txt
pip3 install -e .
```

2. Run the tests (no simulator needed)

```
python3 -m pytest tests/ -v
```

Fourteen tests, all passing, in well under a second.

3. Dry-run the pipeline (no simulator, no LLM)

```
python3 src/main.py \
    prompt "Patrol the perimeter loop twice at 15 metres" \
    mock-llm \
    dry-run
```

This prints the planner's JSON and the validator's result, then stops before the executor stage.

4. Run with SITL

Terminal 1 start the simulator:

```
cd ~/PX4-Autopilot
HEADLESS=0 make px4_sitl gz_x500
```

Wait for the `pxh>` prompt and for the startup script to report it finished. If you want the Gazebo window itself (rather than just the headless server), open a second terminal once `pxh>` is up and run `gz sim -g` to attach a GUI client to it.

Terminal 2 run the pipeline:

```
cd omokai-robotics

# Perimeter patrol, two loops, mock planner
python3 src/main.py \
    prompt "Patrol the perimeter loop twice at 15 metres" \
    mock-llm

# Square pattern, mock planner
python3 src/main.py \
    prompt "Fly a 40 metre square pattern and return to launch" \
    mock-llm

# Replay a previously validated mission file proves the executor
# is deterministic, since this doesn't touch the planner at all
python3 src/main.py \
    mission-file missions/patrol_perimeter.json

# Real local LLM instead of the mock (see "Adding a real LLM" below)
python3 src/main.py \
```

```
prompt "Patrol the perimeter loop twice at 15 metres"
```

Each run writes a log to `logs/run-<mission_id>.log` and the validated mission JSON to `missions/<mission_id>`.

Demo video

`demo/patrol_loop_demo.webm` shows the perimeter patrol prompt executing end-to-end in Gazebo. This recording uses `mock-llm` for a deterministic, repeatable flight path; the real Ollama-backed planner is fully implemented and produces valid missions from the same prompt (see "Adding a real LLM" below), but its output varies run to run since it's a local generative model, so the mock planner was used to guarantee a clean recording.

Quick start Docker

1. Install Docker, if you don't have it

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
# log out and back in (or 'newgrp docker') for the group change to apply
```

2. Build and run

```
cd omokai-robotics
docker compose -f docker/docker-compose.yml up
```

First build compiles PX4 from source inside the container expect 15-20 minutes, not seconds. Later runs reuse the cached image and start in seconds. The `px4-sitl` container runs Gazebo Harmonic (`gz_x500`); the `pipeline` container waits for its healthcheck, then runs the perimeter patrol demo. A successful run ends with `Mission complete.` and a log file under `logs/`.

Custom prompt

```
docker compose -f docker/docker-compose.yml run pipeline \
  python3 src/main.py \
    prompt "Fly a 40 metre square pattern and return to launch" \
    mock-llm \
    connection-url udp://:14540
```

Real LLM inside Docker

Ollama runs on the host, not as a compose service. The compose file maps `host.docker.internal` to the host gateway on the `px4-sitl` service (it has to go there, not on `pipeline` `extra_hosts` can't be combined with `network_mode: service:...`) and points `OLLAMA_HOST` at it by default, so `pipeline` can reach an ollama serve running on your machine with no extra setup **provided Ollama isn't bound to loopback only**:

```
OLLAMA_HOST=0.0.0.0:11434 ollama serve
```

Ollama's default bind (127.0.0.1 only) is invisible to any container connections get refused outright, not just slow. Drop `mock-llm` to use the real planner:

```
docker compose -f docker/docker-compose.yml run pipeline \
  python3 src/main.py \
    prompt "Fly a 40 metre square pattern and return to launch" \
    connection-url udp://:14540
```

If you've changed any Python source or requirements.txt, rebuild first. `docker compose run` does not rebuild automatically without an explicit build it will quietly run whatever was built last, which can look exactly like a normal run against stale code:

```
docker compose -f docker/docker-compose.yml build pipeline
```

`OLLAMA_HOST` and `OLLAMA_MODEL` are plain environment variables, not secrets override either if Ollama runs elsewhere or you want a different model:

```
OLLAMA_MODEL=llama3.1:70b docker compose -f docker/docker-compose.yml run pipeline ...
```

Example prompts

Patrol the perimeter loop twice at 15 metres	<code>patrol_perimeter.json</code>	loops
Fly a 40 metre square pattern and return to launch	<code>square_pattern.json</code>	single pass
Patrol the perimeter three times at 20 m	generated at runtime	loops
Go up to 10 metres and return	generated at runtime	single pass

Project structure

```
omokai-robotics/
src/
  schemas/mission.py Pydantic mission schema, shared by every module
  llm_planner/planner.py prompt -> raw JSON, Ollama or mock
  validator/validator.py schema parse + safety checks
  executor/executor.py deterministic MAVSDK runner
  main.py CLI entrypoint
tests/
  test_validator.py validator unit tests
  test_executor_mock.py mock-based executor tests
missions/ validated mission JSON, plus demo fixtures
logs/ per-run executor logs
docker/
  Dockerfile pipeline image
  Dockerfile.px4 + Gazebo Harmonic, built from source
  docker-compose.yml px4-sitl + pipeline services
  docker-compose.gui.yml override to show the Gazebo GUI
scripts/run_demo.sh one-shot native demo script
ARCHITECTURE.md design rationale and unimplemented-challenge notes
SOURCES.md attribution for every external library or pattern used
```

Adding a real LLM (local, via Ollama no API key, no cost)

1. Install Ollama and pull a model:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.1
ollama serve
```

No root access? Use the redistributable tarball from a user-writable directory instead:

```
mkdir -p ~/.local/ollama && cd ~/.local/ollama
curl -fL -o ollama.tar.zst \
  "https://github.com/ollama/ollama/releases/latest/download/ollama-linux-amd64.tar.
  zst"
tar zstd -xf ollama.tar.zst
OLLAMA_MODELS=~/.local/ollama/models ~/.local/ollama/bin/ollama serve &
~/.local/ollama/bin/ollama pull llama3.1
```

2. Run without `mock-llm`:

```
python3 src/main.py prompt "Patrol the perimeter loop twice at 15 metres"
```

The planner talks to Ollama by default (OLLAMA_HOST defaults to `http://localhost:11434`, OLLAMA_MODEL to `llama3.1`). `mock-llm` always forces the deterministic mock instead, with no LLM call at all.

3. **Why there's a retry loop instead of relying on strict structured output:** a local open model is noticeably less consistent at producing valid JSON than a hosted model's tool-use mode. The Ollama call uses `format="json"` to bias decoding, then checks the result against the same Mission model the real validator uses. If parsing or that check fails, the specific errors go back to the model as a correction request up to three retries before giving up with a clear error. This isn't a second, looser set of rules sitting in front of the real validator; it's the same model, just giving a local LLM more than one shot at satisfying it.
4. The system prompt tells the model directly that it's proposing a plan and nothing more the actual safety gate is the independent Pydantic validator, not the model's own good behavior.

Troubleshooting

ModuleNotFoundError: No
module named 'src'

Takeoff times out

Validation fails on altitude

Gazebo window never opens

run `pip3 install -e .` in the repo root

SITL isn't running start it first

the prompt asked for something outside
the [2, 30] m range

SITL runs headless by default; set
HEADLESS=0 and, if needed, run `gz sim`
-g separately to attach a GUI client

Can't connect to Ollama

`ollama serve` isn't running, or (Docker only) it's bound to `127.0.0.1` only restart with `OLLAMA_HOST=0.0.0.0:11434`
`ollama serve`

Docker pipeline shows MOCK mode even without `mock-llm`

the cached image predates a code change rebuild with `docker compose -f docker/docker-compose.yml build pipeline`; `docker compose run` never rebuilds on its own

`bind error: Address in use` connecting from a native process while `px4-sitl` also runs in Docker

Docker's port mapping already owns that host port stop the container, or run the pipeline via `docker compose run pipeline ...` instead of natively

`conflicting options: custom host-to-IP mapping and the network mode`

`extra_hosts` can't be set on a service using `network_mode: service:...` set it on the service that owns the network namespace instead